

But it would be better to have a combinator that does this passing along for us. Neither `map` nor `map2` will cut it. We need a more powerful combinator, `flatMap`.

**EXERCISE 6.8**

Implement `flatMap`, and then use it to implement `nonNegativeLessThan`.

```
def flatMap[A,B](f: Rand[A])(g: A => Rand[B]): Rand[B]
```

`flatMap` allows us to generate a random `A` with `Rand[A]`, and then take that `A` and choose a `Rand[B]` based on its value. In `nonNegativeLessThan`, we use it to choose whether to retry or not, based on the value generated by `nonNegativeInt`.

**EXERCISE 6.9**

Reimplement `map` and `map2` in terms of `flatMap`. The fact that this is possible is what we're referring to when we say that `flatMap` is *more powerful* than `map` and `map2`.

We can now revisit our example from the beginning of this chapter. Can we make a more testable die roll using our purely functional API?

Here's an implementation of `rollDie` using `nonNegativeLessThan`, including the off-by-one error we had before:

```
def rollDie: Rand[Int] = nonNegativeLessThan(6)
```

If we test this function with various RNG states, we'll pretty soon find an RNG that causes this function to return 0:

```
scala> val zero = rollDie(SimpleRNG(5))._1  
zero: Int = 0
```

And we can re-create this reliably by using the same `SimpleRNG(5)` random generator, without having to worry that its state is destroyed after it's been used.

Fixing the bug is trivial:

```
def rollDie: Rand[Int] = map(nonNegativeLessThan(6))(_ + 1)
```

6.5 A general state action data type

The functions we've just written—`unit`, `map`, `map2`, `flatMap`, and `sequence`—aren't really specific to random number generation at all. They're general-purpose functions for working with state actions, and don't care about the type of the state. Note that, for instance, `map` doesn't care that it's dealing with RNG state actions, and we can give it a more general signature: